

GitHub User Finder

Product Specification Document

1. Product Overview

GitHub User Finder is a single-page React application that allows users to search any GitHub username and instantly view that user's profile information and public repositories. The app fetches data from the public GitHub REST API with no authentication required.

App Name	GitHub User Finder
Type	Single Page Application (SPA)
Framework	React 19 + TypeScript
Build Tool	Vite 6
Data Source	GitHub REST API v3 (public, no auth)
Deployment	Static hosting (Vercel / Netlify)

2. Core Features

2.1 Username Search

The primary interaction of the app. A user types a GitHub username into a search input and submits the form to trigger an API lookup.

- Text input with placeholder 'Search GitHub username...'
- Submit via button click or Enter key
- Input is disabled while a search is in progress
- A clear (×) button appears when the input has a value
- Validation: if submitted with empty input, an inline error message appears with role='alert'
- Validation error clears as soon as the user starts typing again
- onSearch callback receives the trimmed username value

2.2 User Profile Card

Displayed after a successful API response. Shows the GitHub user's public profile information.

- Avatar image (from `avatar_url`) with alt text '[login]'s avatar'
- Display name (name field) — only rendered if not null
- Username as a clickable link to GitHub profile (@login format)
- Bio text — only rendered if not null
- Location with icon — only rendered if not null
- Three stats displayed in a grid: Public Repos, Followers, Following
- Follower/following counts formatted using `formatNumber` utility (e.g. 1200 → 1.2K)
- Card animates in with a fade-up on mount

2.3 Repository List

Displayed below the user profile card. Shows up to 6 of the user's most recently updated public repositories.

- Section heading: 'Recent Repositories'
- Repositories displayed in a responsive grid layout
- Each repo is a clickable link (opens GitHub repo in new tab)
- If `repos` array is empty, shows: 'No public repositories found.'

2.4 Repository Item

Each individual repository card in the grid.

- Repository name (monospace font)
- Description text — if null, shows fallback: 'No description provided'
- Language badge with color dot — only rendered if language is not null
- Star count with star icon
- Entire card is a link with `aria-label`: 'Repository: [name]'

2.5 Loading State

Shown while any API request is in progress.

- Skeleton loading UI — placeholder shapes for avatar, name, bio, stats, and repo cards
- Pulsing animation on the skeleton elements
- Container has `aria-label`='Loading'
- Replaces the main content area while loading

2.6 Error State

Shown when an API request fails for any reason.

- Error container has role='alert'
- 404 response: shows 'User "[username]" not found on GitHub.'
- Non-404 server error (e.g. 500): shows 'Something went wrong. Please try again.'
- Network failure (fetch throws): shows 'Network error. Please check your connection.'
- A 'Try Again' button calls the reset function to clear the error and return to empty state

2.7 Empty State

Shown on initial load before any search has been performed.

- Search icon with message: 'Search for a GitHub user to get started'
- No user card, no repos, no error displayed

3. User Flows

3.1 Successful Search Flow

- User opens the app — empty state is shown
- User types a valid GitHub username (e.g. 'torvalds') into the search input
- User clicks 'Search' or presses Enter
- Loading skeleton appears immediately
- App fetches user data from: GET `https://api.github.com/users/{username}`
- App fetches repos from: GET `https://api.github.com/users/{username}/repos?sort=updated&per_page=6`
- Loading state disappears, UserCard and RepoList render with fetched data

3.2 User Not Found Flow

- User types a non-existent username and submits
- API returns 404
- Error message shown: 'User "[username]" not found on GitHub.'
- User can click 'Try Again' to reset back to empty state

3.3 Empty Input Validation Flow

- User clicks Search without typing anything
- No API call is made
- Inline validation error shown: 'Please enter a GitHub username.'

- Error disappears as soon as user starts typing

3.4 Network Error Flow

- Fetch throws a network-level exception
- Error message shown: 'Network error. Please check your connection.'
- Try Again button resets to empty state

4. API Details

The app uses the GitHub REST API v3. No API key or authentication is required for the endpoints used.

User endpoint	GET https://api.github.com/users/{username}
Repos endpoint	GET https://api.github.com/users/{username}/repos?sort=updated&per_page=6
404 handling	Sets error: 'User "[username]" not found on GitHub.'
Non-404 error	Sets error: 'Something went wrong. Please try again.'
Network failure	Sets error: 'Network error. Please check your connection.'
Rate limits	60 requests/hour unauthenticated (GitHub default)

5. State Management

All state is managed inside the useGithubUser custom hook. No external state library is used.

user	GithubUser null — profile data from the API
repos	GithubRepo[] — list of repositories, default []
loading	boolean — true while any fetch is in progress
error	string null — error message, null when none

The hook exposes two actions:

- searchUser(username: string) — triggers the API fetch sequence

- `reset()` — clears all state back to initial values

6. Component Architecture

App.tsx	Root component. Calls <code>useGithubUser</code> hook. Conditionally renders <code>SearchBar</code> , <code>LoadingState</code> , <code>ErrorMessage</code> , <code>UserCard</code> , <code>RepoList</code> , or empty state.
SearchBar.tsx	Controlled input form. Props: <code>onSearch(username)</code> callback, <code>loading</code> boolean. Handles internal validation state.
UserCard.tsx	Displays user profile. Props: <code>user</code> (<code>GithubUser</code>). Conditionally renders <code>name</code> , <code>bio</code> , <code>location</code> .
RepoList.tsx	Props: <code>repos</code> (<code>GithubRepo[]</code>). Renders grid of <code>RepoItem</code> or empty state message.
RepoItem.tsx	Props: <code>repo</code> (<code>GithubRepo</code>). Renders individual repo card. Conditionally renders <code>description</code> fallback and <code>language</code> badge.
ErrorMessage.tsx	Props: <code>message</code> (string), <code>onRetry</code> optional callback. Has <code>role='alert'</code> .
LoadingState.tsx	No props. Pure skeleton UI with <code>aria-label='Loading'</code> .

7. TypeScript Interfaces

GithubUser interface fields:

- `login`: string — GitHub username
- `name`: string | null — display name
- `avatar_url`: string — URL to avatar image
- `bio`: string | null — profile bio
- `location`: string | null — location string
- `public_repos`: number — count of public repos
- `followers`: number — follower count
- `following`: number — following count
- `html_url`: string — link to GitHub profile

GithubRepo interface fields:

- id: number — unique repo ID
- name: string — repository name
- description: string | null — repo description
- stargazers_count: number — star count
- language: string | null — primary language
- html_url: string — link to repository

8. Utility Functions

8.1 formatNumber(num: number): string

Formats large numbers for display in the stats section.

Input < 1,000	Returns number as plain string (e.g. 500 → '500')
Input >= 1,000	Returns formatted with K suffix (e.g. 1200 → '1.2K')
Input >= 1,000,000	Returns formatted with M suffix (e.g. 2500000 → '2.5M')
Exactly 1000	Returns '1.0K'

9. Non-Functional Requirements

- Responsive layout — works on mobile (single column) and desktop (grid)
- Accessible — all interactive elements have aria labels, errors use role='alert'
- No external state library — useState only
- No routing — single page
- Dark theme — GitHub-inspired color palette
- Skeleton loading prevents layout shift during fetch
- All TypeScript strict mode — no implicit any